

Comparative Analysis of Machine Learning Models for Email Spam Classification

Afroja Akter, Anika Tahasin, Pranta Chowdhury, Samiha Mehjabin

Abstract—This study presents a comparative evaluation of text vectorization strategies paired with Multinomial Naive Bayes (MNB) and Logistic Regression for the automated classification of email spam. Using a corpus of 5,728 email records, we examine the performance of Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) feature extraction methods. Our results indicate that the Bag-of-Words approach combined with a Multinomial Naive Bayes classifier achieved the highest overall accuracy of 98.87 percent. In contrast, while TF-IDF configurations yielded high precision (1.00), they suffered from a significantly lower recall (0.58) for the spam class, failing to identify 42 percent of unsolicited messages. This analysis concludes that simpler frequency-based vectorization is superior for this specific task, as it better captures the high-density trigger words characteristic of spam.

Index Terms—TF-IDF, Multinomial Naive Bayes, Bag-of-Words (BoW).

1 INTRODUCTION

The proliferation of unsolicited email communication, commonly referred to as spam, presents a persistent and evolving challenge in digital communication, necessitating the development of robust filtering systems capable of high-speed and high-accuracy classification. Within the domain of Natural Language Processing (NLP), the transition from unstructured text into numerical feature vectors constitutes the most critical phase of the Machine Learning (ML) pipeline. Current methodologies primarily focus on two distinct vectorization strategies: Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF). Technically, the BoW approach constructs a term-frequency matrix representing a Multinomial distribution of tokens to capture raw occurrence density, whereas TF-IDF applies a logarithmic weighting scheme designed to penalize terms that appear frequently across the entire corpus. However, the efficacy of these schemes is highly dependent on the nature of the specific text data, often leading to a significant trade-off between precision and recall. In our preliminary analysis of 5,728 email records, we observed that while weighted schemes like TF-IDF can achieve a perfect precision of 1.00, they often suffer from an unacceptably low recall of 0.58 for the spam class. This operational limitation results in nearly 42% of unsolicited messages successfully bypassing filters to reach a user's inbox. This research identifies that for short-form spam classification, raw feature density serves as a more potent signal than weighted significance, as spam is often characterized by high-density "trigger words." Visual exploratory data analysis via WordClouds has identified that specific high-frequency keywords—including "money," "free," "click," and "information"—serve as class-conditional high-probability features that strongly signal the spam class. To leverage these features, this study utilizes Multinomial Naive Bayes (MNB),

which calculates the posterior probability $P(L \mid \text{features})$ based on discrete feature counts, and Logistic Regression, which is paired with character-level analysis to address obfuscated text patterns. We implement a rigorous preprocessing pipeline, including an 80/20 train-test split and the removal of non-numeric artifacts, to ensure the integrity of our downstream estimators. By evaluating four distinct experimental configurations, this paper determines the optimal balance between feature weighting and classification accuracy. The Core Contributions of this study are structured :

- **Bridging the Density-Significance Gap:** This study addresses the gap where logarithmic weighting in TF-IDF (89.27% accuracy) dilutes the signal of high-impact triggers; we demonstrate that raw feature density in Bag-of-Words captures these anchors more effectively, achieving a peak accuracy of 98.87%.
- **Mitigating Operational Recall Failure:** We quantify the performance limitations of high-precision filtering models, providing a solution to the research gap where 42% of spam messages go undetected by identifying frequency-based vectorization as a superior method for maintaining high recall.
- **Capturing Sub-word Obfuscation:** This research introduces character-level n-gram analysis (range 3–5) to address the gap left by word-level models, successfully identifying targeted spam samples containing common misspellings or obfuscated terms like "v1agra."
- **A Rigorous Benchmarking Framework:** We provide a comprehensive evaluation of four configurations on a 5,728-record corpus, utilizing standardized preprocessing (`random_state=42`) and n-gram expansion to offer a reproducible baseline for high-speed automated spam classification.

3.4 Preprocessing

A rigorous data cleaning pipeline was implemented to ensure the integrity of downstream estimators. Feature selection was restricted to the text and spam columns. Label consistency was enforced by converting the target variable into a numeric format to handle non-numeric artifacts. Subsequently, the dropna method was applied to remove null entries introduced during the conversion process. Finally, the target labels were explicitly cast to integer types using `astype(int)` to prevent type mismatch errors during model training.

4 METHODOLOGY

The methodology utilized in this study follows a systematic machine learning pipeline designed to transform raw textual data into a diagnostic classification decision. The process is structured into several discrete phases: model architectural selection, experimental configuration design, and a rigorous training and optimization procedure.

4.1 Model Architecture

Two classification paradigms were selected for their efficacy in high-dimensional text spaces:

The Multinomial Naive Bayes classifier is well-suited for discrete feature count representations commonly encountered in text classification tasks. It is grounded in Bayes' theorem and estimates the posterior probability $P(L | \mathbf{x})$, where L denotes the class label and $\mathbf{x}$ represents the feature vector. The model assumes conditional independence among features given the class label, which enables efficient computation in high-dimensional sparse feature spaces. In this study, MNB leverages term-frequency distributions as the primary discriminative signal for classification. Furthermore, Logistic Regression is a linear discriminative model employed for binary classification. It models the conditional probability of the class label using a sigmoid function applied to a linear combination of input features. To improve robustness against adversarial lexical obfuscation (e.g., intentionally distorted tokens such as "v1agra" or spaced character patterns like "f r e e"), the model is augmented with a character-level feature extractor. This enables the capture of subword morphological patterns, thereby enhancing resilience to noisy and obfuscated textual inputs.

4.2 Hyperparameters

Hyperparameter selection is a crucial aspect of optimizing model performance in relation to data complexity. For the Multinomial Naive Bayes (MNB) classifier, a smoothing parameter of $\alpha = 1.0$ is employed to mitigate zero-frequency issues, while `fit_prior=True` enables the model to estimate class prior probabilities directly from the training data, as detailed in Table 1. The Logistic Regression model is configured with `max_iter=1000` to ensure convergence of the optimization process and `C=1.0` to maintain an appropriate balance between bias and variance through regularization. The corresponding parameter configuration is presented in Table 2. Feature extraction is further controlled through vectorization parameters, where `ngram_range=(1, 2)` is

utilized to capture both unigram and bigram representations, and `min_df=2` is applied to eliminate low-frequency terms. These settings effectively reduce feature sparsity and limit the impact of high-dimensional noise. Hyperparameter configurations establish a balanced trade-off between model generalization, computational efficiency, and classification accuracy.

Parameter	Value	Description
α	1.0	Applies Laplace smoothing to handle zero probabilities.
<code>fit_prior</code>	True	Learns class prior probabilities from the training data.
<code>class_prior</code>	None	Uses data-driven priors instead of user-defined priors.
<code>force_alpha</code>	True	Ensures numerical stability by maintaining the smoothing parameter.

TABLE 1: Naive Bayes Parameter Configuration

Parameter	Value	Description
<code>max_iter</code>	1000	Limits the solver to 1,000 iterations before termination.
<code>solver</code>	lbfgs	Uses the L-BFGS optimization algorithm.
<code>penalty</code>	l2	Applies L2 regularization to penalize large coefficients.
<code>C</code>	1.0	Controls inverse regularization strength; smaller values imply stronger regularization.
<code>fit_intercept</code>	True	Adds a bias (intercept) term to the model.
<code>tol</code>	1×10^{-4}	Defines the convergence tolerance threshold.
<code>multi_class</code>	auto	Automatically selects the appropriate classification strategy.
<code>class_weight</code>	None	Assumes equal importance for all classes.
<code>random_state</code>	None	No fixed seed; affects stochastic solvers.

TABLE 2: Logistic Regression Parameter Configuration

5 TRAINING PROCEDURE

We adopted a standard 80 percent and 20 percent split for the dataset. This yielded 4582 samples for training, 1146 samples for final testing. Secondly, we applied data preprocessing and EDA to prepare our data. Thirdly, we trained our data on Multinomial Naive Bayes and Logistic Regression. The experimental configurations were designed to isolate and evaluate the impact of different vectorization strategies and model settings on classification performance.

- **Configuration 1 (BoW + MNB):** Utilizes `CountVectorizer` with English stop-word removal, combined with a standard Multinomial Naive Bayes classifier. This configuration captures raw term frequency as the primary feature representation.
- **Configuration 2 (TF-IDF + MNB):** Employs `TfidfVectorizer` with unigram features and English stop-word removal. This setup introduces weighted feature importance based on term frequency and inverse document frequency.

- **Configuration 3 (TF-IDF + N-grams + MNB):** Extends the TF-IDF approach by incorporating both unigrams and bigrams using `ngram_range=(1, 2)` and applies frequency pruning with `min_df=2` to reduce noise from rare terms.
- **Configuration 4 (Logistic Regression):** Uses `TfidfVectorizer` with a character-level analyzer (`analyzer='char'`) and an n-gram range of `(3, 5)`, along with `min_df=2`. This configuration is paired with a Logistic Regression classifier to capture sub-word patterns and detect obfuscated spam content.

5.1 Losses

Loss functions define how a model measures error during training and guides the learning process.

Logistic Regression: Logistic Regression utilizes the *Log Loss* (also known as Cross-Entropy Loss) as its objective function. It measures the difference between predicted probabilities and actual class labels. The loss function is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where y_i is the true label and p_i is the predicted probability. The goal is to minimize this loss, thereby improving classification accuracy.

Multinomial Naive Bayes: Multinomial Naive Bayes is based on the *Negative Log-Likelihood* of the data. Instead of directly minimizing prediction error, it maximizes the probability of the observed data given the class labels. The objective can be expressed as:

$$\mathcal{L} = -\sum_{i=1}^N \log P(x_i | y_i)$$

To handle zero probabilities for unseen words, Laplace smoothing is applied using a parameter α .

5.2 Optimizers

Optimization determines how model parameters are learned from the data.

Logistic Regression: Logistic Regression relies on iterative optimization algorithms such as `lbfgs` (Limited-memory BFGS). These methods update model weights over multiple iterations (up to 1000 steps or until convergence) to minimize the loss function. The process involves gradient-based updates, making it computationally more intensive but effective for learning complex decision boundaries.

Multinomial Naive Bayes: In contrast, Multinomial Naive Bayes does not require iterative optimization. Instead, it uses a closed-form analytical solution based on counting word frequencies and estimating probabilities directly from the data. This makes training extremely fast and efficient, as it typically requires only a single pass through the dataset.

5.3 Seeds

To ensure reproducibility of experimental results and maintain the integrity of the 80/20 train-test split, a fixed random seed of `random_state = 42` was employed throughout the preprocessing pipeline. This configuration guarantees consistent data partitioning across runs, enabling reliable evaluation of feature vectorization strategies. The dataset was divided into a training set comprising 4,582 samples and a testing set of 1,146 samples. This controlled setup ensures that any observed variations in performance can be attributed to differences in model architecture rather than stochastic effects arising from data splitting.

5.4 Environment

We used the Google Colab free version for this research. This cloud-based system offers 12 GB RAM and an Intel Xeon processor with two virtual cores and two threads, resulting in a clock speed of around 2.20 GHz. Intel Xeon processor x86 architecture can operate in both 32-bit and 64-bit modes. Python is used in this experiment. Machine learning libraries like Scikit-learn are used to conduct this experiment. Also for visualization matplotlib is used.

6 RESULTS

The experimental evaluation of eight distinct configurations, as summarized in Table 3, demonstrates significant variance in performance based on the selected vectorization and classification strategies. The baseline Bag-of-Words (BoW) model (Experiment 1) achieved the highest peak accuracy of 98.87%, indicating that raw term frequency remains a highly effective signal for this specific corpus. In contrast, the standard TF-IDF approach (Experiment 2) resulted in a reduced accuracy of 89.27%. A detailed error analysis of the confusion matrix for this configuration revealed a critical recall value of 0.58, implying that approximately 42% of spam messages were not detected. Expanding the feature space to include word-level bigrams (Experiment 3) further degraded performance, yielding an accuracy of 85.78%. This decline is likely attributable to increased noise and sparsity in the high-dimensional feature space. To address these limitations, iterative data augmentation techniques were applied in Experiments 4–6. These experiments refined probability distributions for high-impact keywords such as “prize” and “rich,” thereby improving qualitative robustness and detection sensitivity. Experiment 7 introduced a frequency-pruned bigram model using a minimum document frequency constraint (`min_df=2`), which reduced feature noise. Experiment 8 implemented a character-level Logistic Regression model with an n-gram range of `(3, 5)`, providing a specialized mechanism for detecting sub-word obfuscations commonly used in spam (e.g., “v1agra”). These results highlight that simpler frequency-based representations can outperform more complex weighting schemes in specific domains, while hybrid and character-level approaches enhance robustness against adversarial text manipulation.

Exp.	Configuration	Vectorization Strategy	Estimator	Key Metric / Result
1	Baseline BoW	Word Unigrams	Multinomial NB	98.87% Accuracy
2	Baseline TF-IDF	Word Unigrams	Multinomial NB	89.27% Accuracy
3	N-gram Expansion	Word (1,2) Range	Multinomial NB	85.78% Accuracy
4	Initial Augment	TF-IDF + 3 Samples	Multinomial NB	Probability shift observed in classification outputs
5	Heavy Augment	TF-IDF + 25 Samples	Multinomial NB	Broadened detection capability across diverse spam patterns
6	Themed Augment	TF-IDF + 5 Samples	Multinomial NB	Improved detection of “Get Rich” themed spam messages
7	Refined N-gram	Word (1,2), $\min_df=2$	Multinomial NB	Reduced feature noise and improved generalization
8	Char-level Shift	Char (3,5), $\min_df=2$	Logistic Regression	Enhanced robustness against obfuscated spam (e.g., “v1agra”)

TABLE 3: Summary of Experimental Configurations

7 DISCUSSION

The comparative analysis of the eight experimental configurations provides critical insights into the operational trade-offs between feature weighting and classification reliability.

7.1 Limitations

A primary limitation of this research is the recall failure observed in weighted configurations. Specifically, the baseline TF-IDF model achieved a recall rate of only 0.58, indicating that approximately 42% of spam messages bypassed the filter. While the Bag-of-Words approach achieved a peak accuracy of 98.87%, its reliance on high-frequency trigger words such as “money” and “free” makes it vulnerable to adversarial evasion through synonym substitution. Furthermore, the dataset size of 5,728 records may not adequately capture the evolving linguistic complexity and diversity of global spam communications. The manual data augmentation employed in Experiments 4–6, although effective in improving detection for specific themes such as “get rich,” represents a limited and non-scalable strategy when compared to real-world, large-scale spam campaigns. Additionally, the observed accuracy drop to 85.78% when using unrefined n-gram features highlights that increased feature dimensionality can introduce significant noise without appropriate feature selection or pruning techniques.

7.2 Ethical Considerations

The deployment of automated text classification systems raises important ethical concerns related to privacy, fairness, and misuse. These models inherently require scanning user-generated content, such as emails, which introduces challenges related to data privacy, secure handling, and user consent. A key ethical risk is the occurrence of false positives, where legitimate (ham) messages—such as meeting updates or project-related communications—are incorrectly classified as spam, potentially leading to professional or personal consequences. Moreover, reliance on a specific training dataset can introduce algorithmic bias, particularly if certain linguistic styles, dialects, or non-standard forms of English are disproportionately misclassified. This can reduce the fairness and inclusivity of the system. Finally, there exists the risk of adversarial misuse. Techniques developed to improve spam detection may also be exploited by malicious

actors to design more sophisticated and harder-to-detect spam messages, thereby escalating the ongoing arms race between spam detection systems and adversaries.

8 CONCLUSION AND FUTURE WORK

This study evaluated eight experimental configurations to identify the optimal pipeline for automated spam classification. The empirical results demonstrate that the Bag-of-Words (BoW) approach, when paired with Multinomial Naive Bayes, remains the most effective strategy, achieving a peak accuracy of 98.87%. In contrast, TF-IDF-based configurations, while capable of achieving high precision, exhibit a significant recall limitation. The baseline TF-IDF model achieved a recall of 0.58, allowing approximately 42% of spam messages to bypass detection. Additionally, the investigation revealed that unrefined n-gram expansion can introduce noise into the feature space, reducing overall model performance to 85.78%. Future research will focus on addressing these recall limitations through the integration of ensemble learning techniques and Transformer-based architectures to better capture contextual and semantic relationships within text data. Furthermore, expanding the current character-level vectorization approach to incorporate deep learning estimators is expected to improve robustness against sub-word obfuscation techniques commonly used in spam (e.g., altered spellings). Finally, the development of an automated, real-time data augmentation pipeline will be explored to ensure continuous model adaptation and resilience against evolving adversarial spam patterns.

ACKNOWLEDGMENTS

This work is supervised by Noortaz Rezoana, Lecturer, Premier University

REFERENCES

- [1] A. Dasgupta et al., “Enhanced Multinomial Naive Bayes Method for Spam E-mail/SMS Text Detection Using TF-IDF Vectorizer,” *American Journal of Mathematical and Computer Modelling*, 2024.
- [2] E. Triana et al., “Improved Spam Email Detection Performance Based on Naive Bayes Approach with TF-IDF Vectorizer,” *Journal of Artificial Intelligence and Engineering Applications*, 2025.
- [3] S. M. Sadid et al., “Spam Filtering Using Multinomial Naive Bayes and TF-IDF,” *Ultimatics*, 2022.
- [4] S. Rapacz et al., “Fast Selection of Machine Learning Classifiers for Spam Filtering,” *Electronics*, 2021.

- [5] F. Jáñez-Martino et al., "Classifying Spam Emails Using Clustering and Topic-Based Methods," *Applied Soft Computing*, 2023.
- [6] Z. Ye, "Comparative Study of Logistic Regression and Naive Bayes Using Apache Spark," *Transactions on Computer Science*, 2024.
- [7] S. M. M. Hossain et al., "Content-Based Spam Detection Using N-gram Models," 2021.
- [8] J. D. Ndibwile et al., "Low Time Complexity Spam Detection Using Logistic Regression," *IJACSA*, 2021.
- [9] B. K. Dedetürk et al., "Spam Filtering Using Logistic Regression with Artificial Bee Colony Optimization," *Applied Soft Computing*, 2020.
- [10] R. Fatima et al., "Optimized Spam Classification Using Ensemble Methods," *Wireless Personal Communications*, 2024.
- [11] A. Dasgupta and M. Mehr, "Benchmarking Classifiers on SMS and Email Datasets," 2023.
- [12] Mathur and Purohit, "Evaluation of Naive Bayes, SVM, and Random Forest for Spam Detection," 2022.
- [13] Gaddamanugu, "Comparative Study of Machine Learning Algorithms for Spam Detection," 2023.
- [14] Abid et al., "Spam Detection Using Frequency-Based Representations," 2023.
- [15] Pandey et al., "Random Forest with TF-IDF for SMS Spam Detection," 2023.
- [16] Marcos and Pacatang, "TF-IDF-Class Variance for Improved Classification," 2024.
- [17] Redondo-Gutierrez et al., "Malicious Spam Detection Using Logistic Regression and TF-IDF," 2023.
- [18] Al-Besher et al., "Topic-Aware TF-IDF for Large-Scale Text Classification," 2024.
- [19] Mohammed and Ahmed, "Combining TF-IDF with GloVe for Spam Detection," 2023.
- [20] Abdelhamid et al., "Hybrid TF-IDF and GPT-Based Spam Detection," 2024.